

人工智能引论

聚类分析

北京邮电大学人工智能引论教研团队



划分型聚类：
K-means聚类

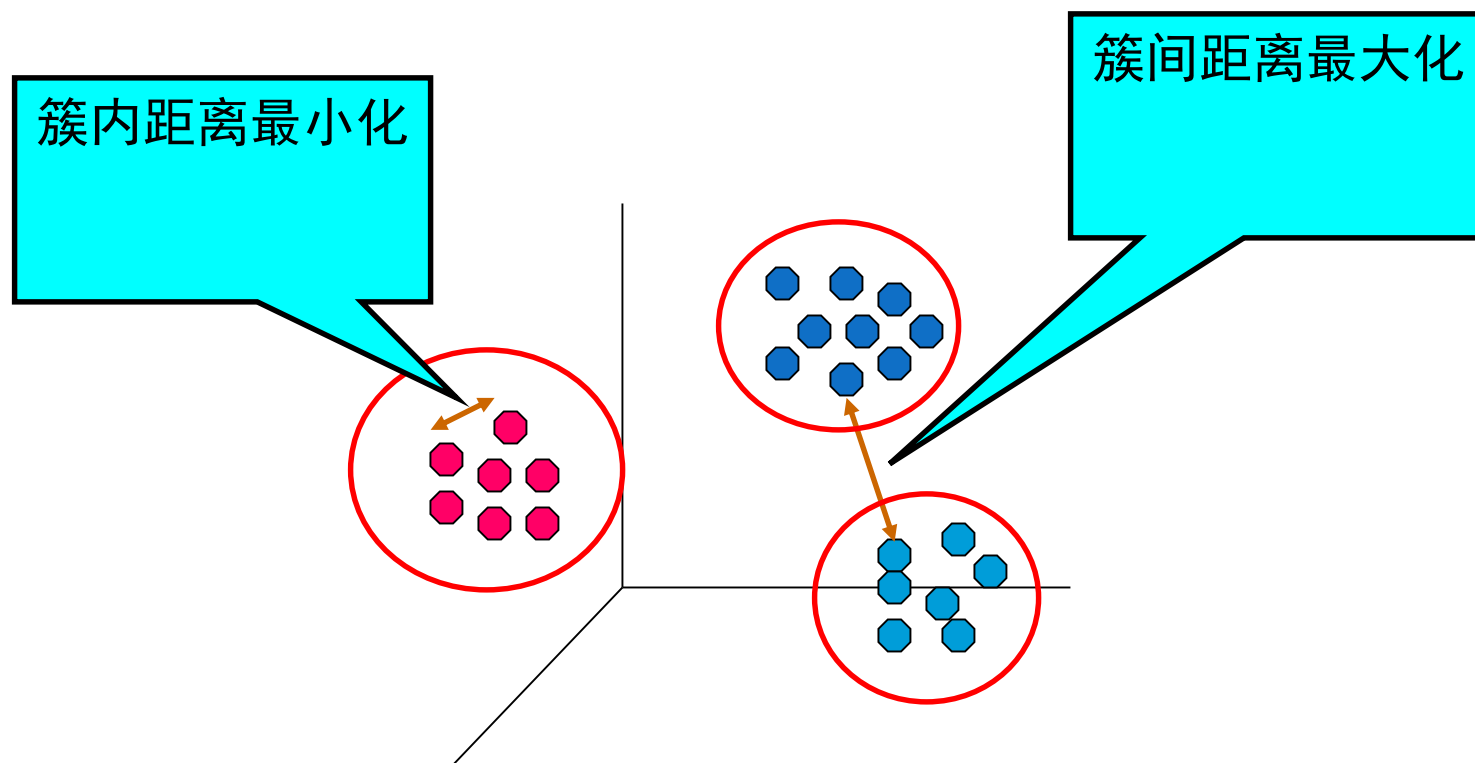
密度聚类：
DBSCAN聚类

层次聚类：
凝聚型层次聚类算法

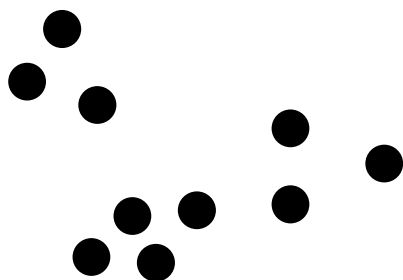
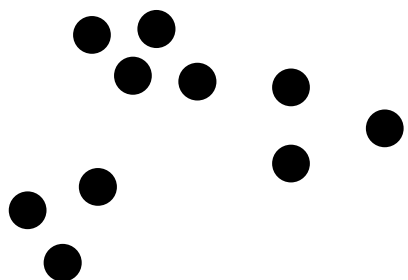
什么是聚类分析



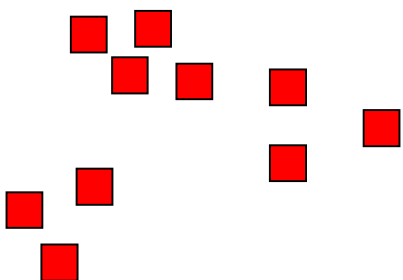
- 查找对象簇，使簇中的对象彼此相似(或相关的)，而与其他簇中的对象不同(或不相关的)



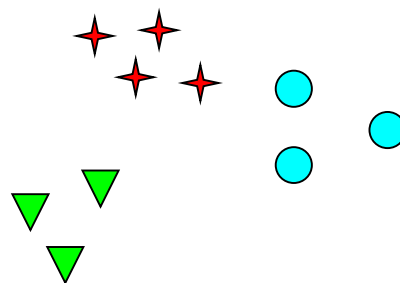
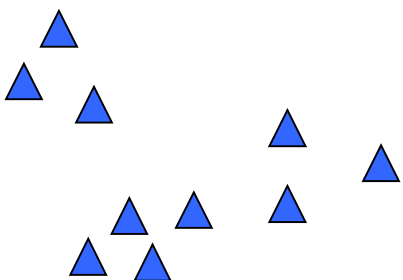
划分成簇的不同方法



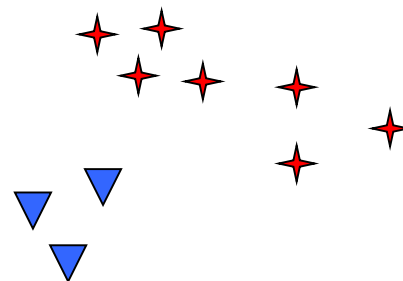
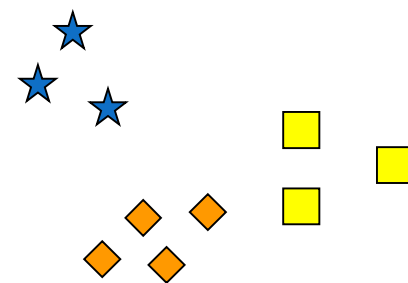
有多少个簇



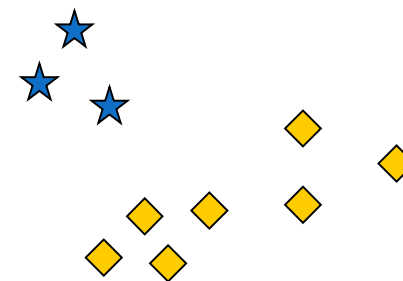
两个簇



六个簇



四个簇



× 层次的与划分的

- +划分的：将数据对象划分为互不重叠的子集(簇)，使每个数据对象正好位于一个子集中
- +层次的：是嵌套簇的集合，组成一棵树，除了叶节点外，树中每个节点都是其子女的并，而树根是包含所有对象的簇。

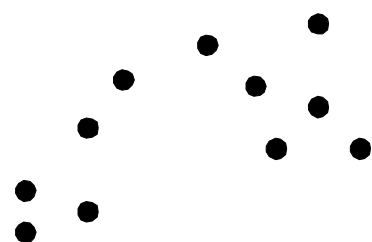
× 互斥的、重叠的与模糊的

- +互斥的：每个对象都被指派到单个簇。
- +重叠的：一个对象同时属于多个簇
- +模糊聚类：每个对象以一个0和1之间的隶属权值属于每个簇

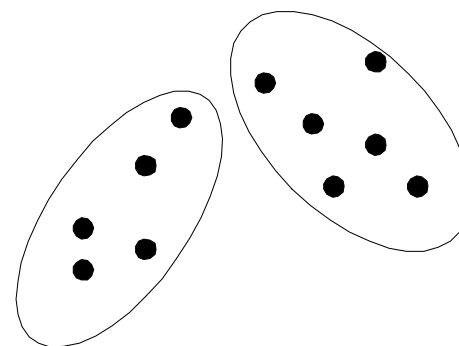
× 完全的与部分的

- +完全聚类：将每个对象指派到一个簇。部分聚类与此相反。

划分型聚类算法效果示例

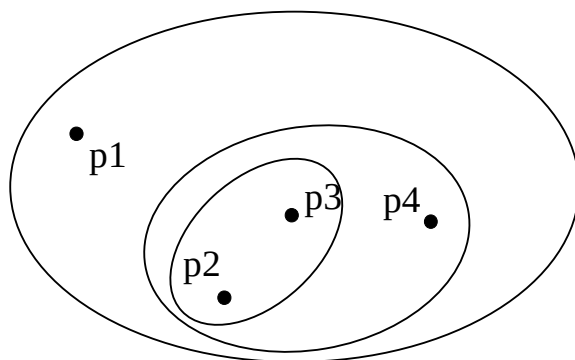


原始点

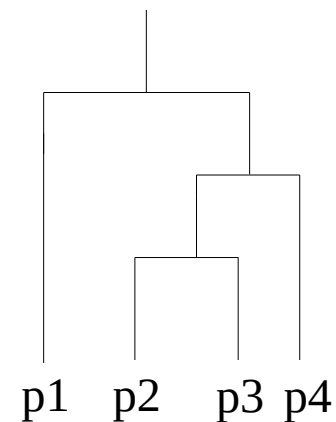


划分聚类

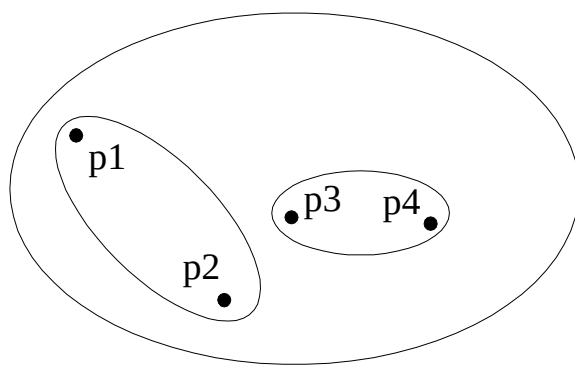
层次型聚类算法效果示例



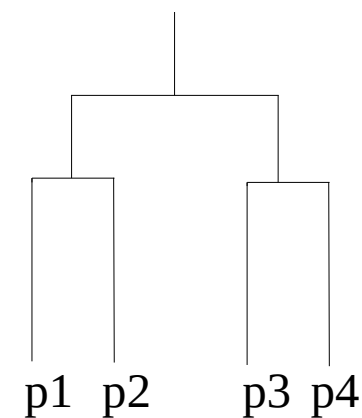
层次聚类效果1



层次聚类效果1对应的树结构



层次聚类效果2

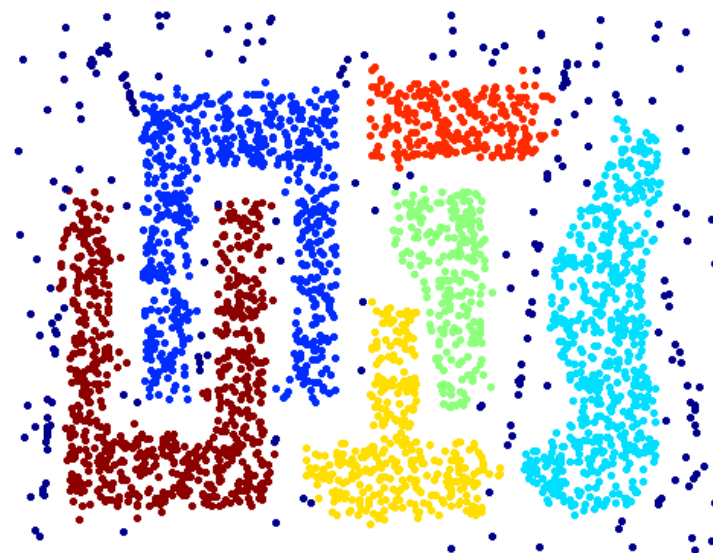


层次聚类效果2对应的树结构

密度型聚类算法效果示例



Original Points



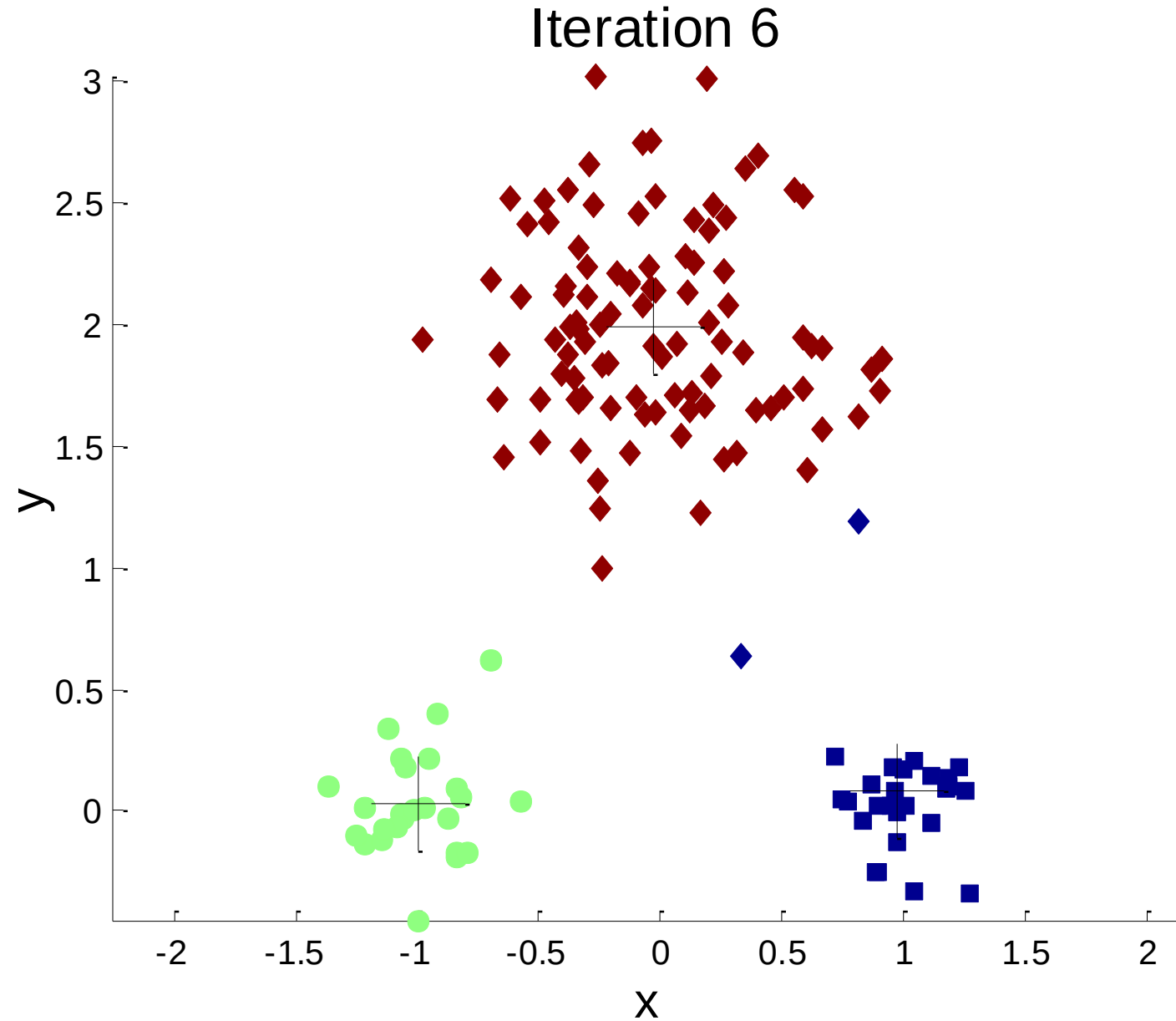
Clusters

K-means 聚类算法

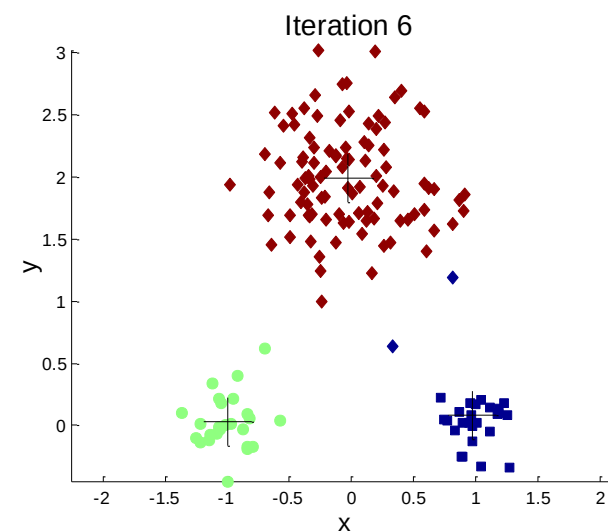
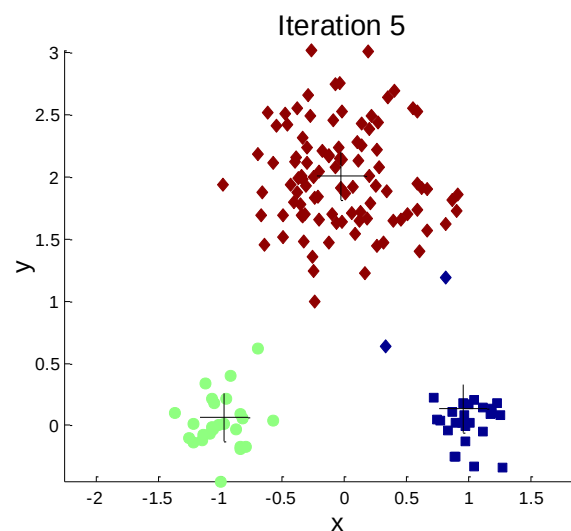
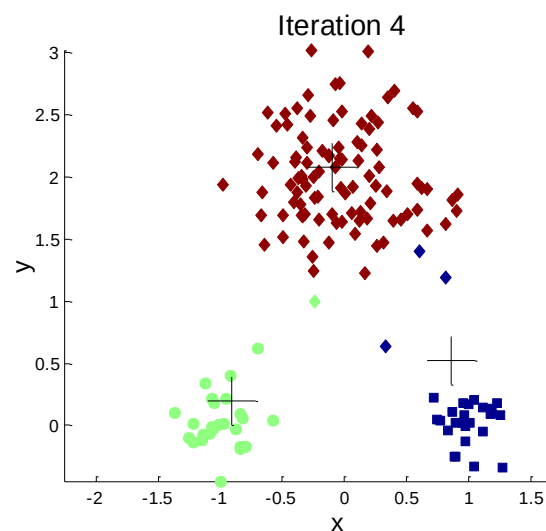
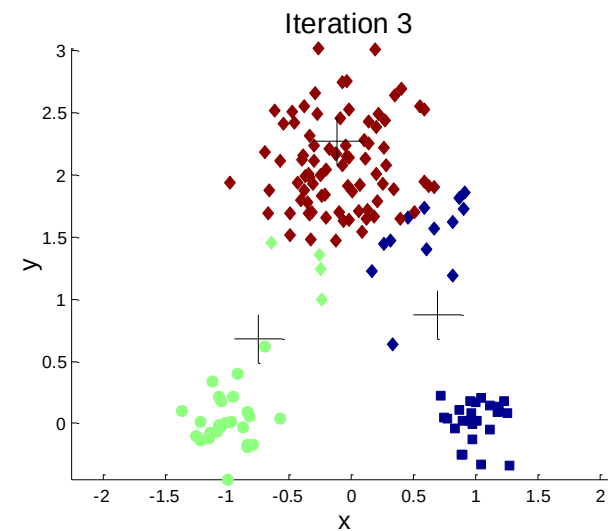
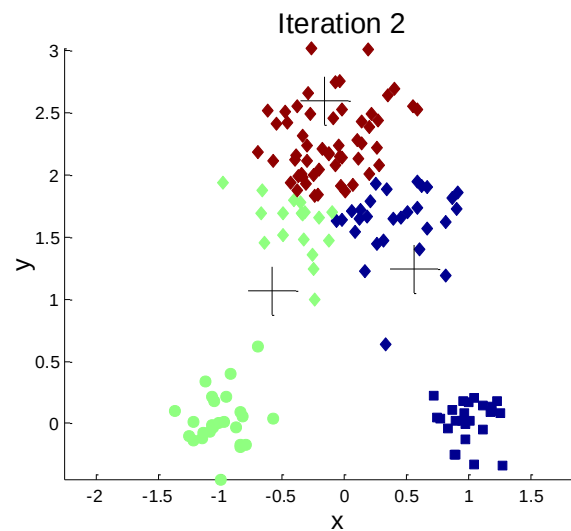
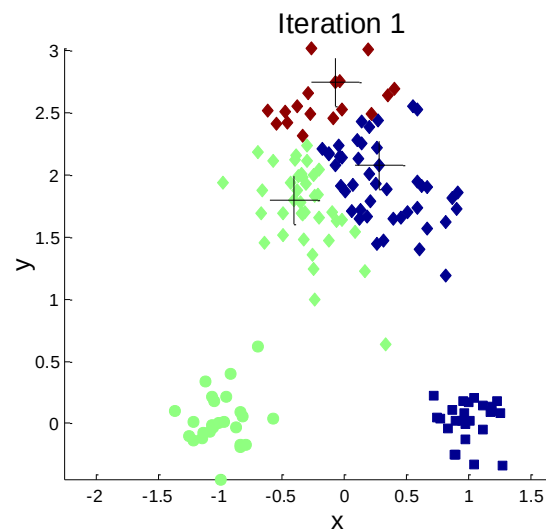


- × 属于划分聚类
- × 必须指定簇的数量 K
- × 每个簇都与一个质心(中心点)相关联。
- × 每个点都分配给具有最近的质心的簇
- × 根据指派到簇的点, 更新每个簇的质心
- × 重复指派和更新步骤, 直到簇不发生变化或者质心不再发生变化

K-means聚类算法流程



K-means聚类算法演示流程



K-means Clustering – 质心

- ✖ 初始质心往往是随机抽取的。
- ✖ “相似性” 以欧氏距离来衡量, 也可以采用余弦相似性、曼哈顿距离等。
- ✖ “重新计算每个簇的质心” —— 因为质心会随着数据邻近性度量和聚类目标不同而改变。



K-means Clusters 目标函数

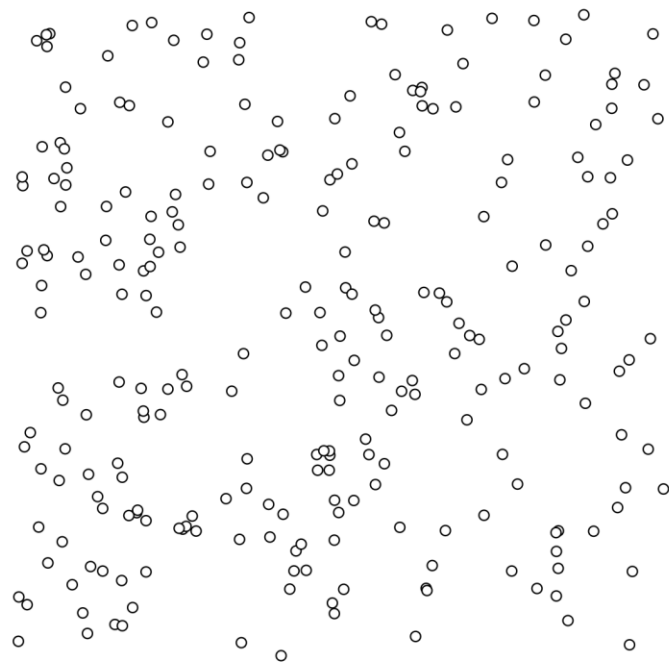
- 最常见的度量聚类的目标函数是误差平方和 (SSE)
- 计算每个数据点的误差，即它到最近质心的欧式距离
- 然后计算误差平方并求和。

$$SSE = \sum_{i=1}^K \sum_{x \in C_i} dist^2(m_i, x)$$

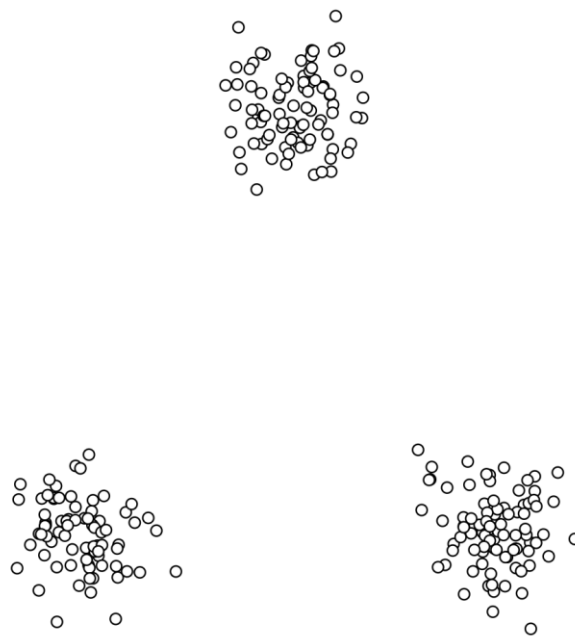
- x 是簇 C_i 中的一个数据点， m_i 是簇 C_i 的质心
- K 均值需要最小化 SSE
- 思考：对于三个二维点 $(1, 1)$, $(2, 3)$, $(6, 2)$ ，其质心是多少？
- 质心是 $((1+2+6) / 3, (1+3+2) / 3) = (3, 2)$

K-means 算法演示网址

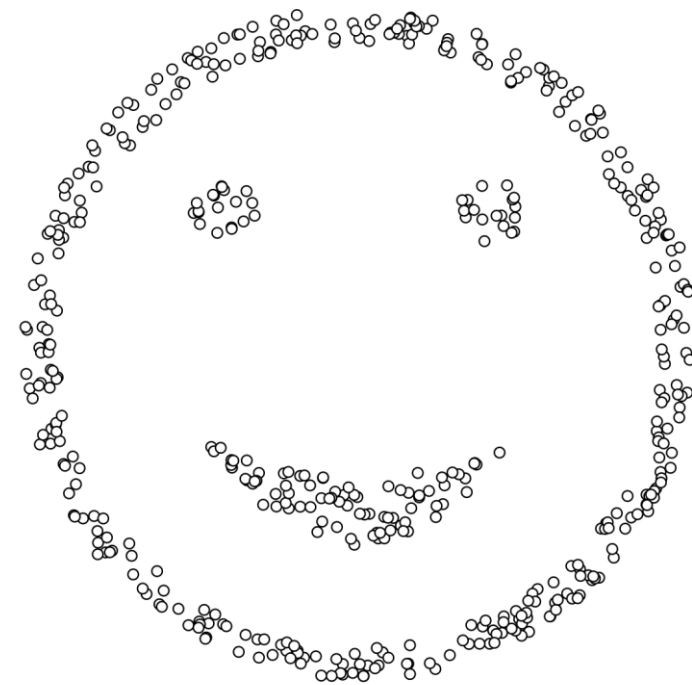
✖ <https://www.naftaliharris.com/blog/visualizing-k-means-clustering/>



均匀分布



高斯混合分布



笑脸分布



How to pick the initial centroids?

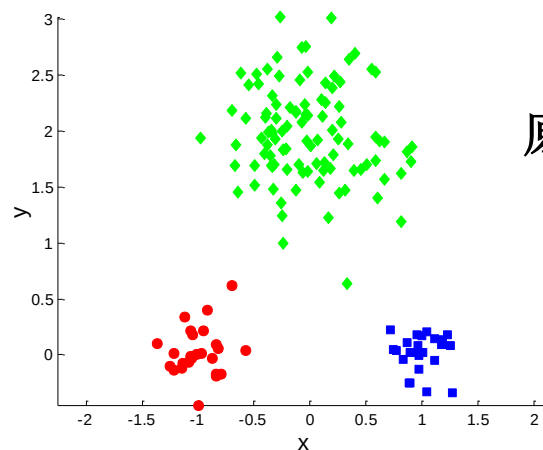
I'll Choose

Randomly

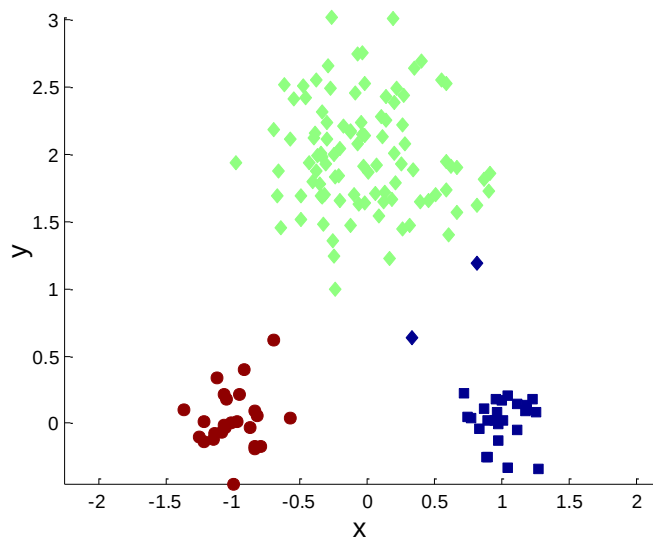
Farthest Point

Restart

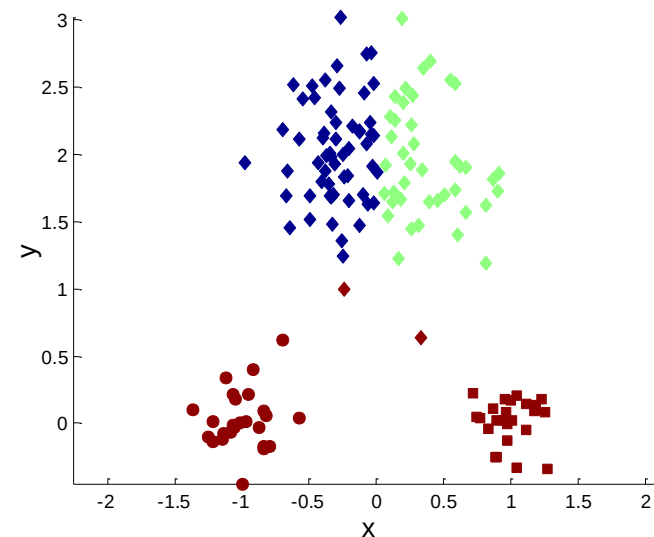
K-means的不同聚类结果



原始数据点



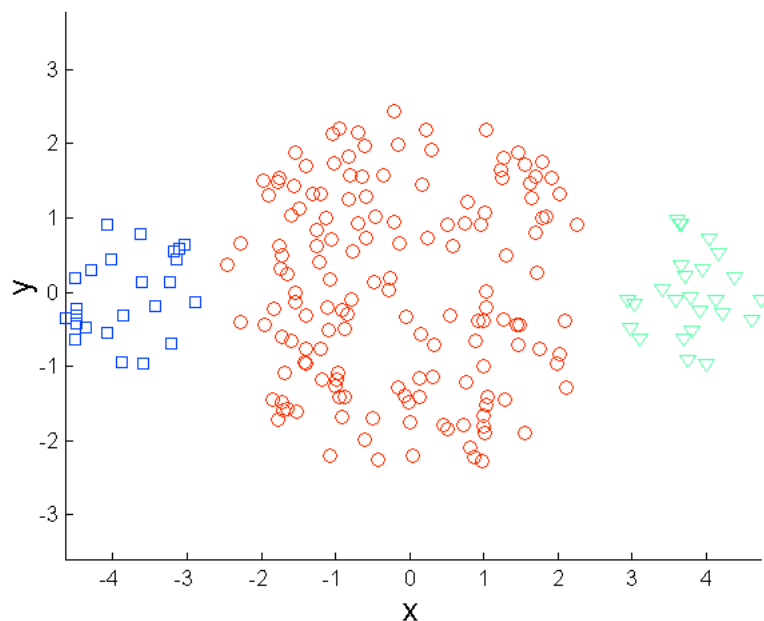
最优化聚类结果



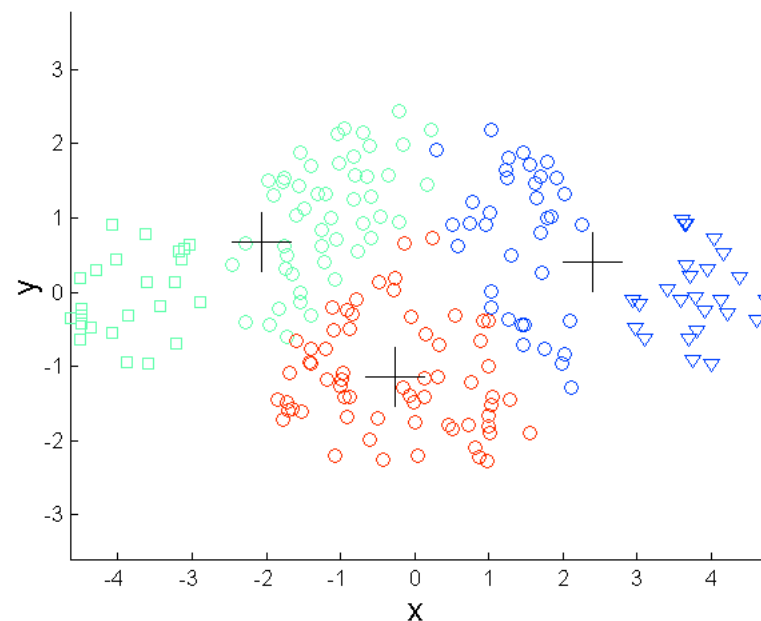
次优化聚类结果

- ✖影响K-means运行效果的限制因素
 - +规模
 - +密度
 - +非球形形状
- ✖当数据包含异常值时，K-means就会出现问题。

K-means限制——数据规模



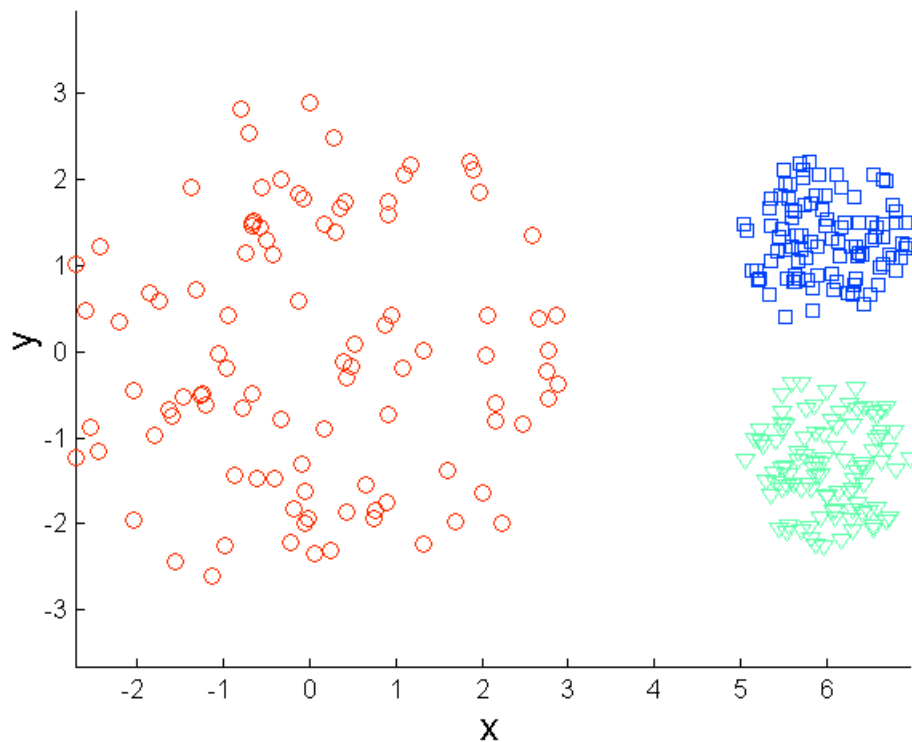
Original Points



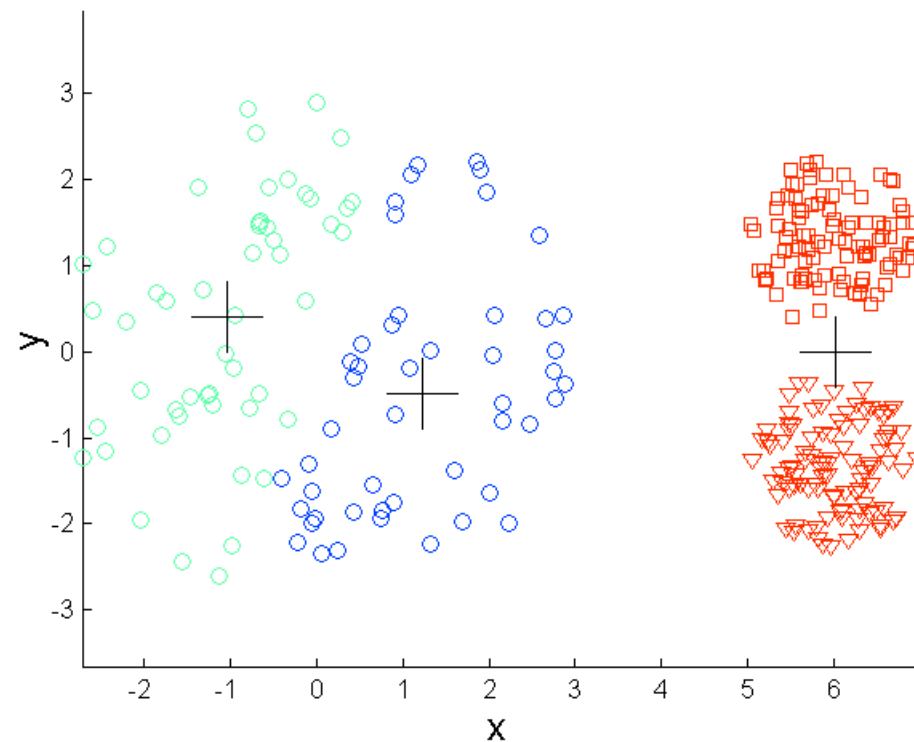
K-means (3 Clusters)

K-means算法需要对每个数据点进行多次距离计算和簇分配，数据规模越大，计算量越大。随着数据规模的增加，算法的初始化质心选择对最终结果的影响也越大，可能导致局部最优解。

K-means限制——密度



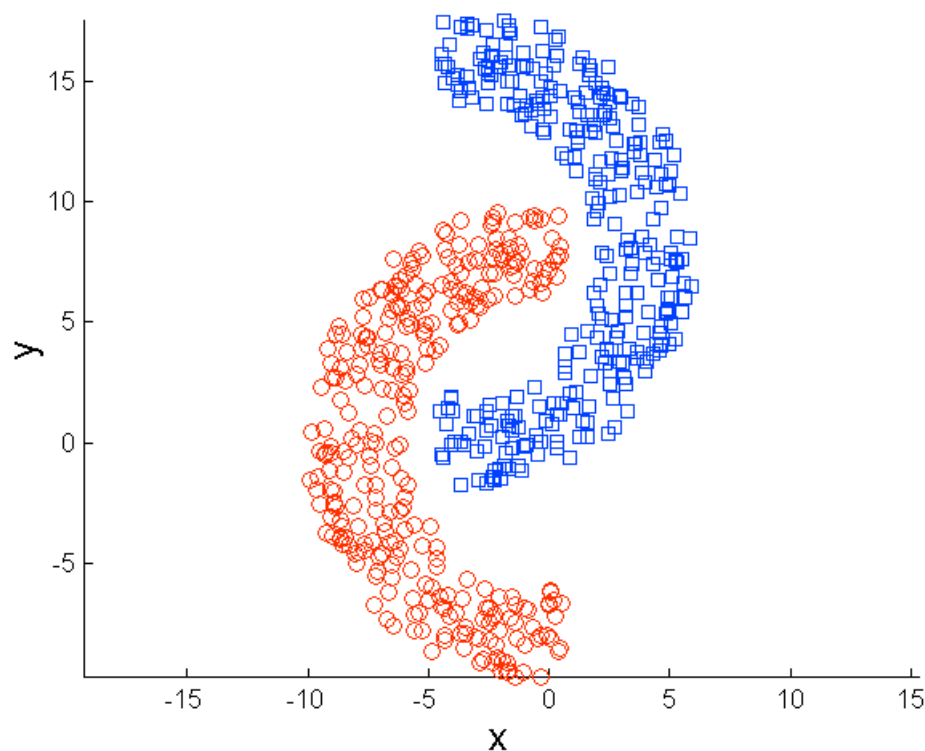
Original Points



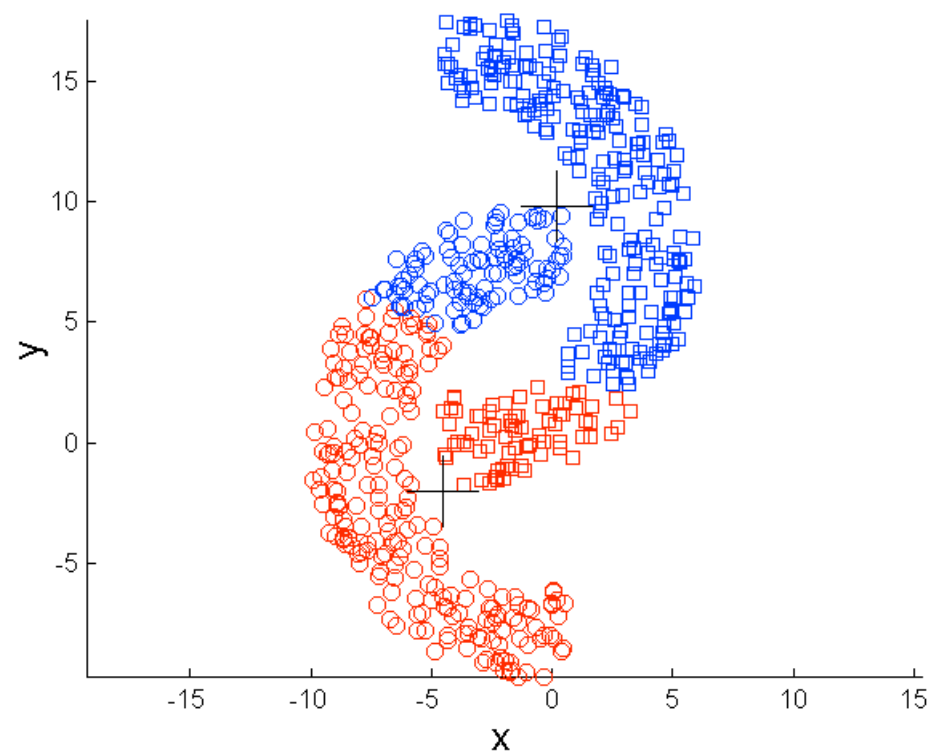
K-means (3 Clusters)

K-means假设数据是均匀分布的，因此K-means可能会错误地将低密度区域的数据点分配到高密度区域的簇中，导致簇的划分不准确。K-means的目标函数是最小化误差平方和（SSE），这会导致算法倾向于生成大小相似的簇，而忽略密度差异。

K-means限制——非球形



Original Points



K-means (2 Clusters)

K-means假设簇是球形的，因此对于非球形分布的数据（如环形、流形或任意形状的簇），K-means无法正确识别簇的边界，导致聚类结果不准确。

初始质心选择优化方法

- ✖ 随机选择第一个点，或取所有点的质心作为第一个点
- ✖ 对于每个后继质心，选择离已经选取过的初始质心最远的点
- ✖ 但问题是可能选中离群点，而不是稠密区域中的点

✖使用平方误差标准时，离群点可能过度影响所发现的簇。即当存在离群点时，结果簇的质心可能不如没有离群点时那样有代表性，并且SSE也比较高。正因为如此，提前发现离群点并删除它们是有用的。

✖然而，应当意识到有一些聚类应用，不能删除离群点。当聚类用来压缩数据时，必须对每个点聚类。在某些情况下(如财经分析)，明显的离群点(如不寻常的有利可图的顾客)可能是最令人感兴趣的点。优化方法：

- + 聚类前删除离群点
- + 后处理识别离群点
- + 删除很小的簇

- ✖ K均值适用范围较为广泛
- ✖ 但对于非球形簇、不同尺寸和不同密度的簇处理效果较差
- ✖ 容易受离群点的影响

DBSCAN算法是一种基于密度的聚类算法：

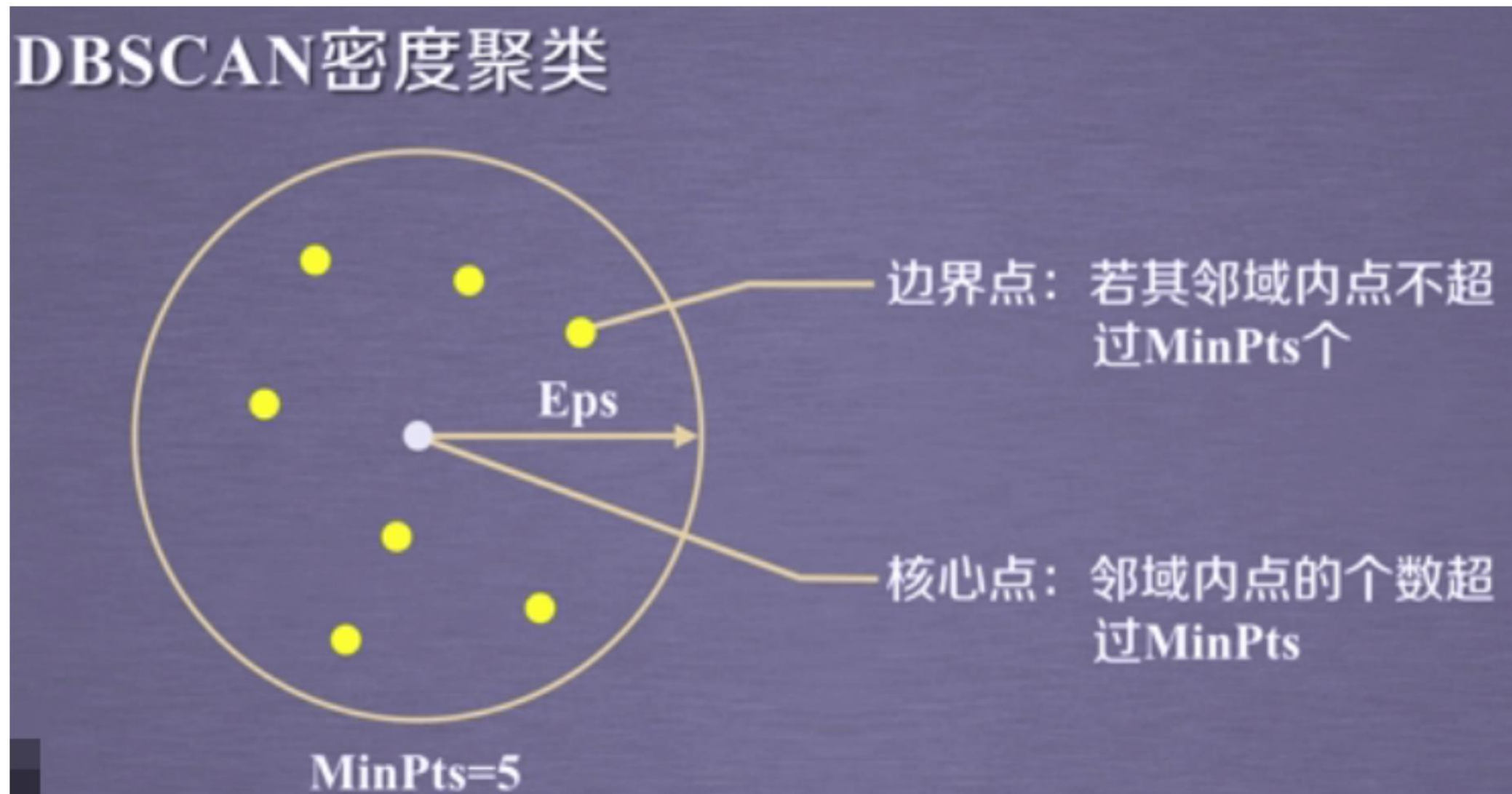
1. 聚类的时候不需要预先指定簇的个数
2. 最终的簇的个数不确定

邻域的概念：对于任意样本 i 和给定距离 e ，样本 i 的 e 邻域是指所有与样本 i 距离不大于 e 的样本集合。

DBSCAN算法将数据点分为三类：

1. 核心点：在半径 Eps 内含有超过 $MinPts$ 数目的点。
2. 边界点：在半径 Eps 内点的数量小于 $MinPts$ ，但是落在核心点的邻域内的点。
3. 噪音点：既不是核心点也不是边界点的点

DBSCAN: 核心点、边界点和噪声点

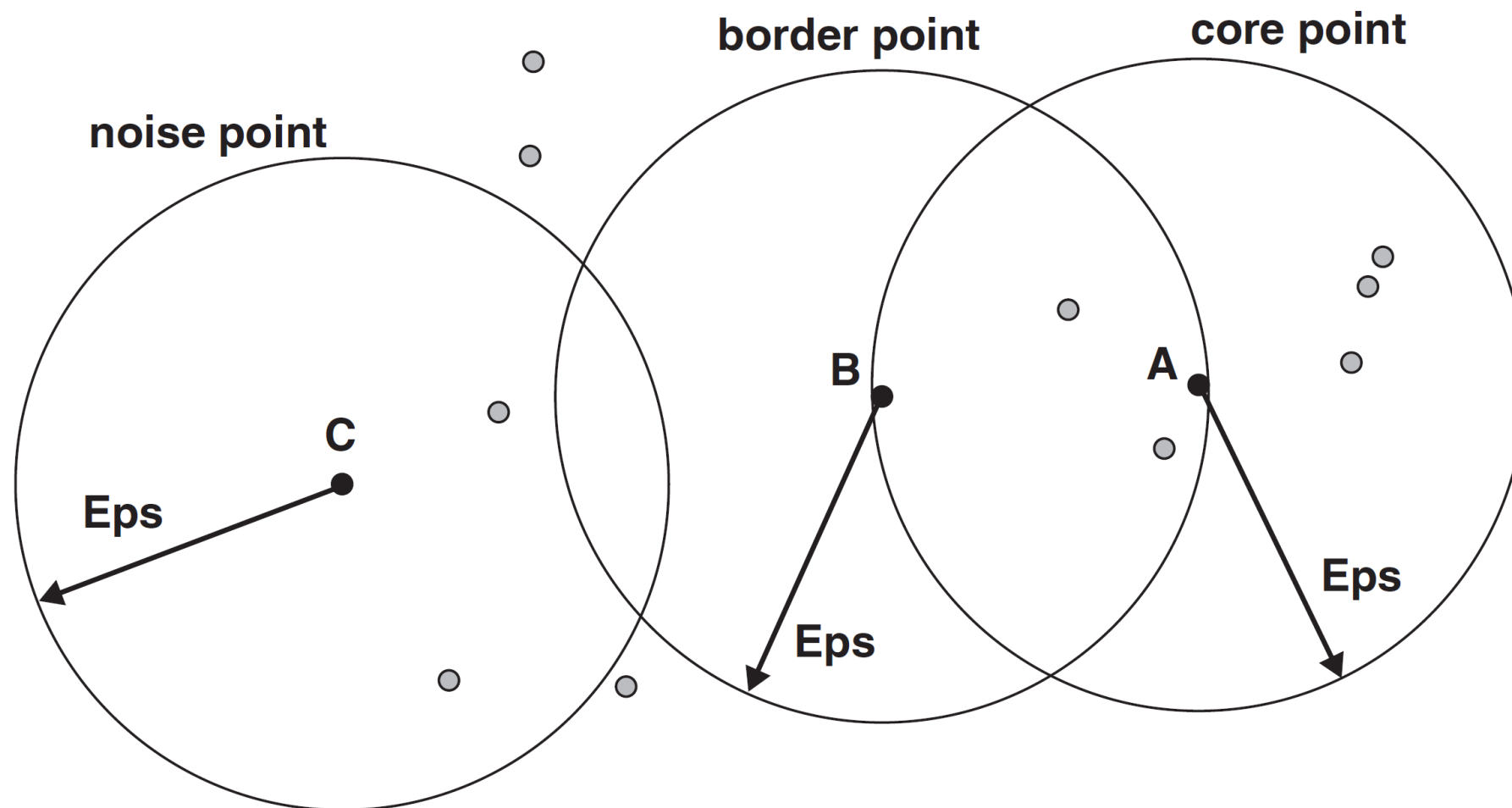


DBSCAN: 核心点、边界点和噪声点



北京邮电大学
Beijing University of Posts and Telecommunications

MinPts = 7



DBSCAN聚类算法

- 定义 1 直接密度可达：给定一个对象集合 D ，如果 p 是在 q 的 ε -邻域内，而 q 是一个核心对象，我们说对象 p 从对象 q 出发是直接密度可达的。

例如，在图1中， $\varepsilon=1\text{cm}$ ， $\text{MinPts}=5$ ， q 是一个核心对象，对象 p 从对象 q 出发是直接密度可达的。

- 定义 2 密度可达的：如果存在一个对象链 $p_1, p_2, \dots, p_n$ ， $p_1=q$ ， $p_n=p$ ，对 $p_i \in D$ ， $(1 \leq i \leq n)$ ， p_{i+1} 是从 p_i 关于 ε 和 MinPts 直接密度可达的， q 是一个核心点，则对象 p 是从对象 q 关于 ε 和 MinPts 密度可达的。

例如，在图2中， $\varepsilon=1\text{cm}$ ， $\text{MinPts}=5$ ， q 是一个核心对象， p_1 是从 q 关于 ε 和 MinPts 直接密度可达， p 是从 p_1 关于 ε 和 MinPts 直接密度可达，则对象 p 从对象 q 关于 ε 和 MinPts 密度可达的（间接密度可达）。

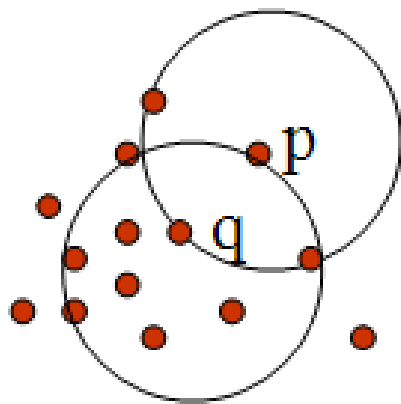


图1 直接密度可达

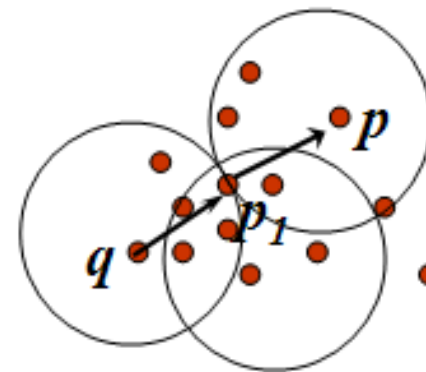


图2 密度可达

- DBSCAN通过检查数据集中每个对象的 ε -邻域来寻找聚类。如果一个点 p 的 ε -邻域包含多于MinPts个对象，则创建一个 p 作为核心对象的新簇。然后，DBSCAN反复地寻找从这些核心对象直接密度可达的对象，这个过程可能涉及一些密度可达簇的合并。当没有新的点可以被添加到任何簇时，该过程结束。具体如下：

DBSCAN算法描述

算法 DBSCAN

输入：包含 n 个对象的数据库，半径 ε ，最少数目MinPts。

输出：所有生成的簇，达到密度要求。

1. REPEAT
2. 从数据库中抽取一个未处理过的点；
3. IF 抽出的点是核心点 THEN找出所有从该点密度直达和密度可达的数据点，形成一个簇
4. ELSE 抽出的点是边缘点(非核心点)，跳出本次循环，寻找下一点；
5. UNTIL 所有点都被处理；

DBSCAN聚类算法

下面给出一个样本事务数据库（见左表），对它实施DBSCAN算法。
根据所给的数据通过对其进行DBSCAN算法，以下为算法的步骤（设 $n=12$ ，用户输入 $\varepsilon=1$ ，MinPts=4）

样本事务数据库

序号	属性 1	属性 2
1	1	0
2	4	0
3	0	1
4	1	1
5	2	1
6	3	1
7	4	1
8	5	1
9	0	2
10	1	2
11	4	2
12	1	3

DBSCAN算法执行过程示意

步骤	选择的点	在 ε 中点的个数	通过计算可达点而找到的新簇
1	1	2	无
2	2	2	无
3	3	3	无
4	4	5	簇 C_1 : {1, 3, 4, 5, 9, 10, 12}
5	5	3	已在一个簇 C_1 中
6	6	3	无
7	7	5	簇 C_2 : {2, 6, 7, 8, 11}
8	8	2	已在一个簇 C_2 中
9	9	3	已在一个簇 C_1 中
10	10	4	已在一个簇 C_1 中,
11	11	2	已在一个簇 C_2 中
12	12	2	已在一个簇 C_1 中

聚出的类为{1, 3, 4, 5, 9, 10, 12}, {2, 6, 7, 8, 11}

DBSCAN聚类算法



算法执行过程:

步骤	选择的点	在 ϵ 中点的个数	通过计算可达点而找到的新簇
1	1	2	无
2	2	2	无
3	3	3	无
4	4	5	簇 C_1 : {1, 3, 4, 5, 9, 10, 12}
5	5	3	已在簇 C_1 中
6	6	3	无
7	7	5	簇 C_2 : {2, 6, 7, 8, 11}
8	8	2	已在簇 C_2 中
9	9	3	已在簇 C_1 中
10	10	4	已在簇 C_1 中,
11	11	2	已在簇 C_2 中
12	12	2	已在簇 C_1 中

第1步, 在数据库中选择一点1, 由于在以它为圆心的, 以1为半径的圆内包含2个点 (小于4), 因此它不是核心点, 选择下一个点。

第2步, 在数据库中选择一点2, 由于在以它为圆心的, 以1为半径的圆内包含2个点, 因此它不是核心点, 选择下一个点。

第3步, 在数据库中选择一点3, 由于在以它为圆心的, 以1为半径的圆内包含3个点, 因此它不是核心点, 选择下一个点。

第4步, 在数据库中选择一点4, 由于在以它为圆心的, 以1为半径的圆内包含5个点, 因此它是核心点, 寻找从它出发可达的点 (直接可达4个, 间接可达2个: 10也是一个核心点, 4直接密度可达有1 3 5 10, 通过10间接密度可达的有9和12), 聚出的新类{1, 3, 4, 5, 9, 10, 12}, 选择下一个点。

第5步, 在数据库中选择一点5, 已经在簇1中, 选择下一个点。

第6步, 在数据库中选择一点6, 由于在以它为圆心的, 以1为半径的圆内包含3个点, 因此它不是核心点, 选择下一个点。

第7步, 在数据库中选择一点7, 由于在以它为圆心的, 以1为半径的圆内包含5个点, 因此它是核心点, 寻找从它出发可达的点, 聚出的新类{2, 6, 7, 8, 11: 第二个簇 C_2 中没有第二个核心点了}, 选择下一个点。

第8步, 在数据库中选择一点8, 已经在簇2中, 选择下一个点。

第9步, 在数据库中选择一点9, 已经在簇1中, 选择下一个点。

第10步, 在数据库中选择一点10, 已经在簇1中, 选择下一个点。

第11步, 在数据库中选择一点11, 已经在簇2中, 选择下一个点。

第12步, 选择12点, 已经在簇1中, 由于这已经是最后一点所有点都已处理, 程序终止。

DBSCAN算法迭代可视化

<https://www.naftaliharris.com/blog/visualizing-dbscan-clustering/>

DBSCAN算法中

- ϵ (epsilon)设置过大，不同簇可能会合并为一个簇，导致无法区分密度不同的区域；
- ϵ (epsilon)设置过小，则只有非常接近的点会被归为同一簇，可能导致大量点被标记为噪声。
- minPoints设置过大，只有密度非常高的区域才会被识别为簇，可能会忽略低密度但有意义的簇；
- minPoints设置过小，即使是密度较低的区域也会被识别为簇，可能会生成过多小簇并将噪声点误认为是簇的一部分。

DBSCAN算法迭代可视化



What kind of data would you like?

Uniform Points

Gaussian Mixture

Smiley Face

Density Bars

Packed Circles

Pimpled Smiley

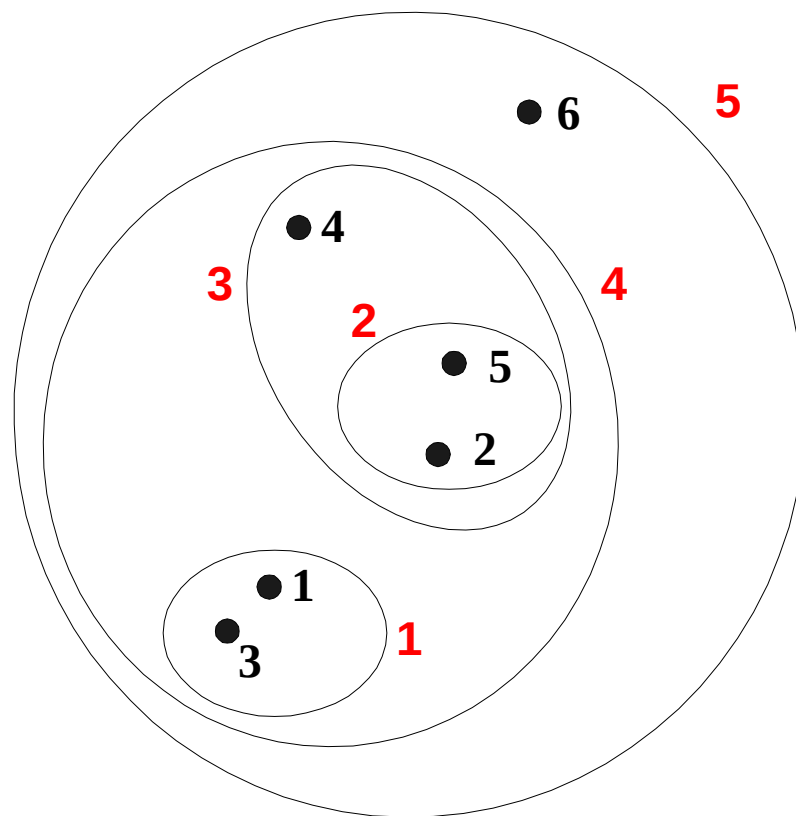
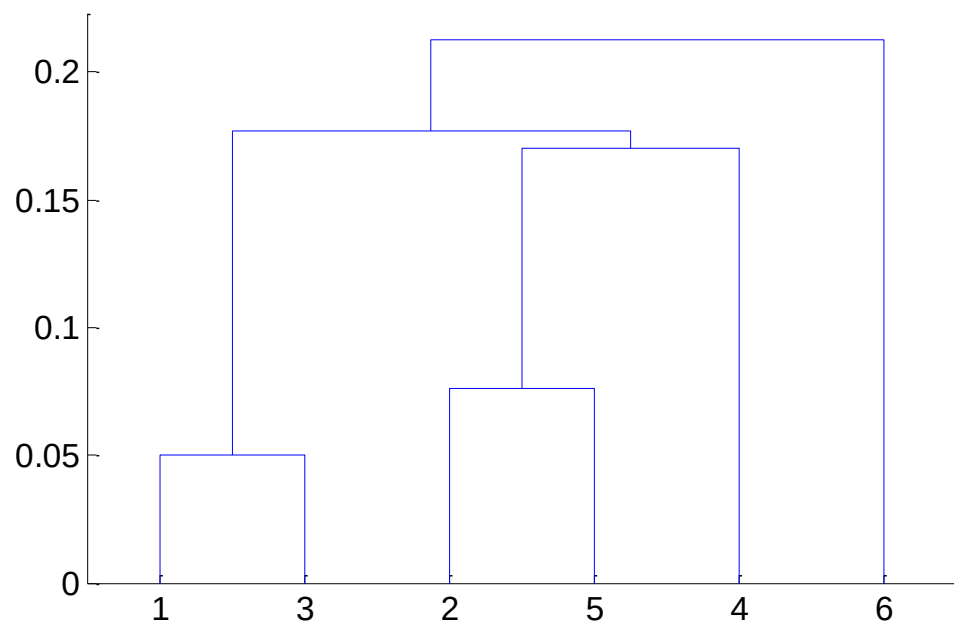
DBSCAN Rings

Example A

Restart

凝聚型层次聚类法

- 生成一组按层次树组织的树状图
- 树状图记录合并或分裂的次序



层次聚类的两种主要类型：

- ✓ **凝聚型：**
 - ✓ 从点作为单独的簇开始
 - ✓ 在每一步中，合并最近的一对簇，直到只剩下一个簇(或k个簇)为止
- ✓ **分裂型：**
 - ✓ 从一个包含所有点的某个簇开始
 - ✓ 在每个步骤中分裂一个簇，直到每个簇包含一个单点簇为止

传统的层次聚类算法使用邻近度或距离矩阵
一次合并或拆分一个集群

凝聚型层次聚类法-算法步骤

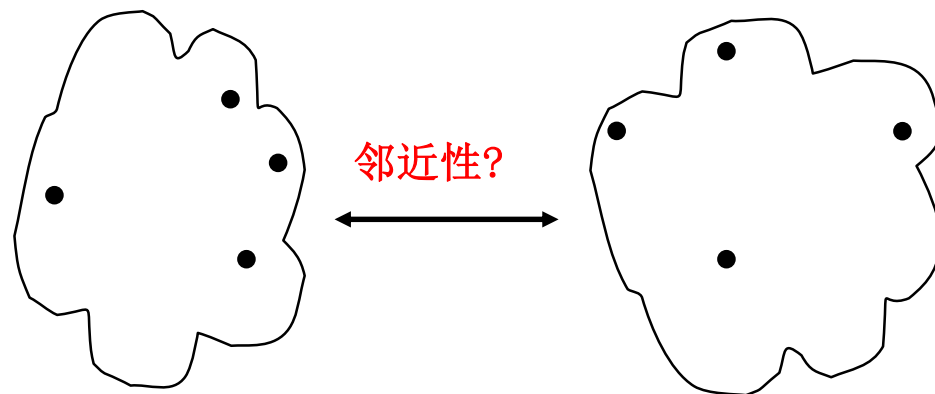


基本算法步骤

1. 计算邻近度矩阵
2. 每个数据点成为一个簇
3. Repeat
4. 合并最接近的两个簇
5. 更新邻近度矩阵
6. Until 只剩下一个簇为止

- ✖ 关键操作是计算两个簇的邻近度
- ✖ 定义邻近度的不同方法区分了不同的算法效果

定义簇之间的邻近性



MIN

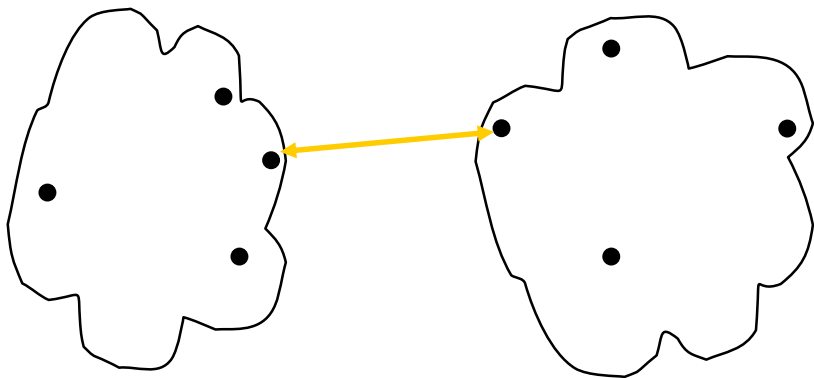
MAX

组平均

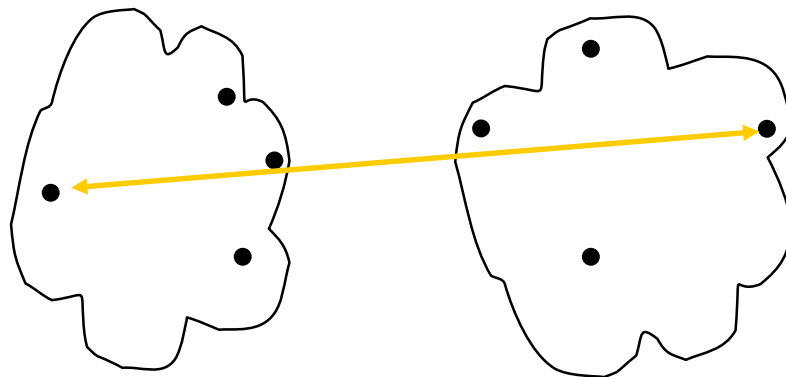
	p1	p2	p3	p4	p5	.	.
p1						.	.
p2							
p3							
p4							
p5							
.							
.							
.							

Proximity Matrix

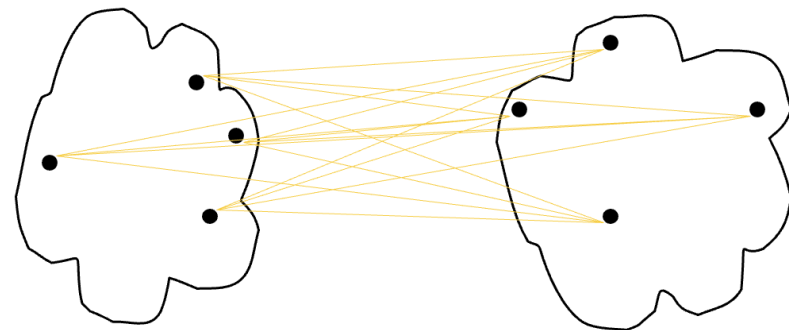
定义簇之间的邻近性



MIN: 定义簇的邻近度为不同的簇的两个最近点之间的邻近度
或表示为不同的节点子集中两个节点之间的最短边
又叫做单链方式



MAX: 定义不同簇中两个最远的点之间的邻近度作为簇的邻近度
或者使用图的术语，不同的结点子集中两个结点之间的最长边
又叫做全链



组平均: 邻近度定义为取自不同簇的所有点对邻近度的平均值（平均边长）

凝聚型层次聚类法的问题

- ✗ 一旦决定合并两个簇，就无法撤消当前操作，阻碍了局部最优向全局最优进化
- ✗ 无法最小化全局目标函数
- ✗ 计算量和存储需求代价较大
- ✗ 问题：
 - + 如果在某一步没有选择合适的分割点，可能会导致聚类结果的质量较低。
 - + 大数据集并不能很好地分解。